

Product Requirement Document (PRD)

Balatro Tactical Companion Engine

REAL-TIME HYPERGEOMETRIC DRAW ENGINE & DYNAMIC STATE MACHINE

Document Version: 1.0	Target Environment: Antigravity Pipeline	Author: Systems Architecture Group	Status: Implementation Ready
-----------------------	--	------------------------------------	------------------------------

1. Executive Summary & Core Objective

In the poker-based roguelike *Balatro*, strategic success depends on predicting draw probabilities from a highly fluid, structurally altered deck. Standard statistical models break down because the baseline deck parameters change constantly—players add custom card duplicates, alter ranks, delete elements, change suits via Tarot manipulations, or leverage modifiers like Wild and Stone cards.

The objective of this project is to build an independent, zero-friction **Balatro Tactical Companion App** utilizing an asynchronous data infrastructure. This application will track real-time deck allocations, parse hand configurations, and compute high-fidelity draw success metrics via a robust Hypergeometric distribution layout. By isolating the data schema from static poker conventions, the platform ensures that upcoming modifiers, card duplication, and rule-breaking enhancements remain functionally integrated from the first compilation step.

2. Core Architectural Principles & Data Schema

To prevent architectural bottlenecks when integrating modifiers later, cards are modeled as complex, self-contained structures rather than simple index strings or scalar pairs. Every asset within the user's collection or active deck inherits from a unified, extensible data object blueprint.

2.1 Card Priming State Schema

Every entity inside the tracking inventory must conform strictly to the following structure:

```
{
  "id": "string (uuid-v4)",
  "base_rank": "string (2-10, Jack, Queen, King, Ace)",
  "base_suit": "string (Spades, Hearts, Diamonds, Clubs)",
  "enhancement": "string (None, Bonus, Mult, Wild, Glass, Steel, Stone, Lucky, Gold)",
  "edition": "string (None, Foil, Holographic, Polychrome)",
  "seal": "string (None, Gold, Red, Blue, Purple)",
  "is_destroyed": "boolean"
}
```

2.2 Active Environment State Arrays

The engine manages three core localized pools of objects during execution:

- **Remaining Deck Pool (N):** A highly dynamic collection of all active card objects remaining inside the current deck layout, which scales dynamically as cards are added or erased.

- **Current Hand Dock:** Up to 8 visible card entities mapped directly to the active screen view.
- **Target Discard Selection Vector (n):** Sub-elements selected from the hand dock by the user, indicating cards slated for immediate removal, which defines the exact size of the incoming draw sample.

3. Mathematical Engine Specifications

The math framework computes outcomes using a deterministic hypergeometric configuration model. It completely discards traditional approximations in favor of true combinatorial permutations, ensuring precision even when evaluating duplicate structures.

3.1 The Hypergeometric Probability Model

The exact probability of drawing exactly k target cards when pulling a sample of size n from a deck of size N containing K viable items is calculated via:

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

Because completing a complex hand structure in *Balatro* generally requires hitting *at least* a specified number of outs, the system must evaluate and display the cumulative success metric ($P(X \geq k)$):

$$P(X \geq k) = \sum_{i=k}^{\min(n, K)} \frac{\binom{K}{i} \binom{N-K}{n-i}}{\binom{N}{n}}$$

Where the combination term $\binom{n}{r}$ is executed computationally via:

$$\binom{n}{r} = \frac{n!}{r! (n-r)!}$$

Boundary Exception Guard Rules

If $k > n$, $k > K$, or $n > N$, the evaluation framework must catch the out-of-bounds error early and return a deterministic **0.00%** success indicator, rather than causing a calculation crash.

4. Functional Requirements & Feature Sets

4.1 Deck Configuration Interface

- **Dynamic Grid Tracking:** Provides a 13×4 layout UI allowing users to increment or decrement individual rank/suit allocations instantly.
- **Global Size Realignment:** Automatically updates the total deck size denominator (N) whenever an item state transitions.
- **Tarot Overlay State:** Allows easy bulk re-assignment of card parameters to mimic in-game Arcana card usage.

4.2 Real-Time Calculator Pipeline

- **Discard Sample Sizing:** Automatically monitors how many cards the user selects for a discard, setting the exact size of the drawn sample (n).
- **Cumulative Metric Feedback:** Computes the specific success probability within a 50ms latency window, providing immediate statistical feedback.
- **Multi-Target Calculations:** Displays success metrics across multiple potential hands simultaneously.

4.3 "Outs" Identification Layer Logic

The engine flags valid deck elements as matching targets (K) based on the user's goal hand. The detection logic operates via specific condition profiles:

Target Hand Type	Required Logic Criteria	Target Group Strategy (v0 Base Deck Validation)
Flush	5 cards matching identical suit parameters.	Scans Remaining Deck Pool; matches when <code>base_suit == target_suit</code> .
Four of a Kind	4 cards matching identical rank parameters.	Scans Remaining Deck Pool; matches when <code>base_rank == target_rank</code> .
Full House	3 cards of Rank A; 2 cards of Rank B.	Evaluates dual-step probabilities based on current missing elements for both targets.
Straight (Outside)	4 sequential cards with open high/low slots.	Scans Remaining Deck Pool for both the immediate upper and lower missing rank items.
Straight (Inside)	4 structural cards missing an interior rank slot.	Scans Remaining Deck Pool exclusively for the single specific rank required to close the gap.

4.4 Modifier Separation Architecture (Future-Proofing Design)

To ensure adding modifiers later is completely seamless, the system handles complex elements by adjusting the **Outs Identification Layer** while leaving the underlying math module untouched:

- **Wild Cards:** Evaluated in the target identification loops by expanding the matching condition to check: `if (base_suit == target_suit || enhancement == "Wild")`. This seamlessly updates the pool value (K) without altering the main hypergeometric calculation logic.
- **Stone Cards:** Handled by explicitly bypassing standard rank and suit validation filters while continuing to count toward the total deck asset volume denominator (N).

5. UI/UX Interface Specification

To match the focused, distraction-free preferences of strategic gameplay, the interface layout is structured around two main operational segments:

LEFT PANEL: INVENTORY CONFIGURATION MATRIX

A dense, intuitive control grid that enables quick state adjustments with zero workflow friction.

- **Matrix Grid:** A 13×4 panel for editing active deck counts instantly.
- **Hand Tracking Dock:** High-contrast buttons to simulate active cards currently held in hand.
- **Action Toggles:** Fast commands to clear arrays, reset to a standard 52-card setup, or trigger bulk Tarot transformations.

RIGHT PANEL: COMPUTATION PANEL

A clean, high-contrast dashboard displaying real-time success metrics and telemetry data.

- **Primary Odds Gauge:** Displays the calculated cumulative success metric ($P(X \geq k)$) in a large, easy-to-read layout.
- **Target Selection Menu:** Interactive toggles to set the desired hand type.
- **Telemetry Feed:** Shows the underlying variables (N , K , n , k) to verify calculation accuracy.

6. Technical Constraints & Non-Functional Requirements

- **Processing Latency:** The mathematical calculation loop must execute in under 50 milliseconds upon any change to the state variables.
- **State Preservation:** The system must automatically cache active session layouts locally to prevent data loss during accidental reloads or pipeline updates.
- **Data Cleanliness:** Separation of concerns must be strictly enforced between the state tracker, the target identification logic, and the core probability engine. This ensures that adding new card variants or modifiers never breaks the application structure.